CART Project Report

Classification and Regression Trees

Wine cultivar classification and synthetic house-price regression

5 November 2026 | Python and scikit-learn

Executive summary

This project implements two separate, reproducible CART workflows. The classification notebook predicts one of three wine cultivars from chemical measurements. The regression notebook predicts simulated house prices in lakhs of rupees. Each workflow compares a full tree, a pre-pruned tree, and a cost-complexity post-pruned tree against a simple baseline.

Model selection uses five-fold cross-validation within the training partition. Held-out test data are used for evaluation and interpretation. Both notebooks execute successfully from start to finish, and their complete datasets are supplied as separate CSV files.

Outcome	Classification	Regression
Dataset	178 wine samples	400 synthetic houses
Training / test rows	133 / 45	300 / 100
Selected model	Post-Pruned Tree	Pre-Pruned Tree
Selected tree size	Depth 3; 4 leaves	Depth 9; 48 leaves
Held-out result	Accuracy 95.6%; macro F1 0.959	RMSE 12.56 lakhs; R^2 0.858

Main conclusion

Both workflows substantially outperform their respective baselines. Pruning offers a way to control complexity, but a model that wins cross-validation need not win on one particular test split. The reported selections follow the validation procedure rather than the test ranking.

Submitted artifacts

Classification_Tree.ipynb — complete classification analysis and saved outputs.

Regression_Tree.ipynb — complete regression analysis and saved outputs.

Classification_Dataset.csv — 13 measurement columns plus Cultivar.

Regression_Dataset.csv — four property features plus Price_lakhs.

CART_Project_Report.pdf — this combined project report.

1. Datasets and preparation

1.1 Wine classification

The classification data come from the Wine dataset bundled with sklearn.datasets.load_wine. There are 178 observations and no missing values or duplicate complete rows in the inspected dataset. The target counts are class_0: 59, class_1: 71, and class_2: 48. These names identify cultivars; they are not quality rankings.

The 13 numeric features are alcohol, malic_acid, ash, alcalinity_of_ash, magnesium, total_phenols, flavanoids, nonflavanoid_phenols, proanthocyanins, color_intensity, hue, od280/od315_of_diluted_wines, and proline. Original feature names and measurement values are preserved.

Classification_Dataset.csv contains 178 rows and 14 columns. Its Cultivar column stores readable class names. The notebook uses integer targets 0, 1, and 2, corresponding to class_0, class_1, and class_2. No row-index column is exported.

1.2 Synthetic house-price regression

Column	Meaning / generation
Area_sqft	Floor area: integer from 600 to 3,500 square feet
Bedrooms	Bedroom count: integer from 1 to 5
Age_years	Property age: integer from 0 to 30 years
Distance_km	Distance: uniform draw from 1 to 25 km, rounded to one decimal
Price_lakhs	Continuous target, generated using the formula below

Before clipping and rounding, price = 15 + 0.035 × Area_sqft + 4 × Bedrooms − 0.6 × Age_years − 1.2 × Distance_km + 15 × I(Distance_km ≤ 5) + noise. Noise follows a normal distribution with mean 0 and standard deviation 5. Prices are clipped to a minimum of 10 lakhs and rounded to two decimals.

Regression_Dataset.csv contains 400 rows and five columns, with no missing values. Generation uses NumPy default_rng(42). The simulated features are independent by design; the dataset is not a record of real housing transactions.

1.3 Preprocessing and export integrity

Both datasets are fully numeric apart from the exported classification label. Decision trees do not require feature scaling. Each CSV preserves the complete dataset and original row order, rather than exporting only a training partition. Reloaded CSV values were checked against the DataFrames created by the notebooks.

2. CART methodology and validation

2.1 Binary splitting and predictions

CART recursively selects a feature and threshold to partition a node into two children. Each split is chosen to reduce weighted child impurity relative to the parent. Binary splitting applies even though the wine problem has three target classes.

For classification, Gini impurity is $G = 1 - \Sigma\, p_k^2$. A pure node has Gini 0. A leaf predicts its majority class and returns its class proportions as probabilities. For regression, squared-error impurity is the mean of $(y_i - \text{mean}(y))^2$ within a node, and each leaf predicts its mean training price.

2.2 Data separation

Both workflows reserve 25% of data with random_state=42. Classification uses stratified train/test sampling and five-fold StratifiedKFold with shuffling. Regression uses a random split and five-fold shuffled KFold. Cross-validation operates only on the training data.

2.3 Models and hyperparameter grids

Approach	Classification	Regression
Full tree	Gini; unrestricted growth	Squared error; unrestricted growth
Pre-pruning	max_depth: 1–6; min_samples_leaf: 1, 2, 4, 8	max_depth: 2, 3, 4, 5, 6, 8, 10, None; min_samples_leaf: 2, 5, 10
Post-pruning	ccp_alpha: 0 plus 61 logarithmic values from 10^{-4} to 1	ccp_alpha: 0 plus 40 logarithmic values from 10^{-2} to 10^{3}
Baseline	Most frequent training class	Mean training price

Pre-pruning restricts growth through depth and leaf-size controls. Cost-complexity post-pruning balances weighted leaf impurity against alpha times the number of leaves. The fixed alpha grids are independent of validation-fold targets; the full-training pruning paths are illustrative.

2.4 Selection and metrics

Classification maximizes mean CV balanced accuracy (average recall across classes). Regression minimizes mean CV MSE; displayed CV RMSE is the square root of mean fold MSE, not mean fold RMSE. Full and tuned trees remain eligible. Exact ties prefer fewer leaves, then lower depth.

Classification reports accuracy, balanced accuracy, macro F1, and log loss. Regression reports MAE, RMSE, and R^2. Accuracy/F1/R^2 are higher-is-better; error metrics and log loss are lower-is-better. Tuning scores can be optimistic because multiple candidates were compared.

3. Classification results

The post-pruned tree was selected with ccp_alpha = 0.0184785. It has depth 3 and four leaves. Its CV balanced accuracy is 91.93%; it correctly predicts 43 of 45 held-out samples.

Model	CV bal. acc.	Test accuracy	Test macro F1	Leaves
Majority baseline	33.3%	40.0%	0.190	—
Full Tree	90.4%	95.6%	0.956	8
Pre-Pruned Tree	91.8%	100.0%	1.000	7
Post-Pruned Tree	91.9%	95.6%	0.959	4

Selected model: test balanced accuracy 95.56%, macro F1 0.959, and log loss 0.175. Both errors are class_0 samples predicted as class_1. Recall is 86.7% for class_0 and 100% for classes 1 and 2.

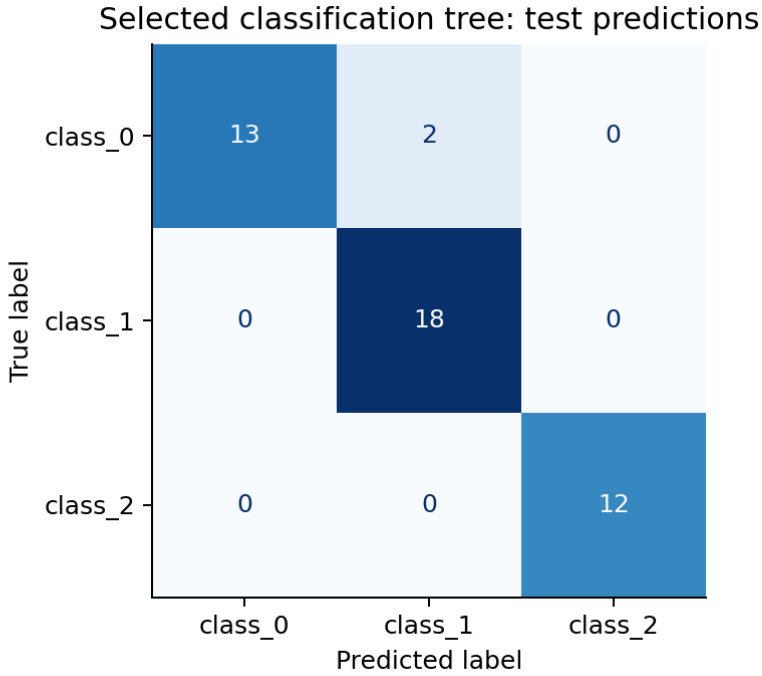


Figure 1. Held-out confusion matrix for the CV-selected classification model.

The pre-pruned tree obtains 100% test accuracy, but its CV balanced accuracy (91.84%) is slightly below the selected tree's (91.93%). This small validation difference is not strong evidence of superiority. The selected model is retained rather than changed after viewing test scores.

The selected tree splits on color intensity, flavanoids, and proline. The notebook includes impurity importance, held-out permutation importance, and readable rules. A hypothetical sample based on training medians, with alcohol 13.2 and proline 850, is predicted as class_0; it has no known true label.

4. Regression results

The pre-pruned tree was selected with max_depth = 10 and min_samples_leaf = 5. Its realized depth is 9 with 48 leaves. CV RMSE is 10.82 lakhs.

Model	CV RMSE	Test MAE	Test RMSE	Test R^2
Mean baseline	32.64	28.34	33.29	-0.000
Full Tree	11.46	9.04	11.44	0.882
Pre-Pruned Tree	10.82	9.77	12.56	0.858
Post-Pruned Tree	11.32	9.92	12.01	0.870

MAE and RMSE are measured in lakhs. The selected model's average absolute test error is 9.77 lakhs, with RMSE 12.56 lakhs and R^2 0.858. Its test RMSE is 62.3% below the mean-price baseline.

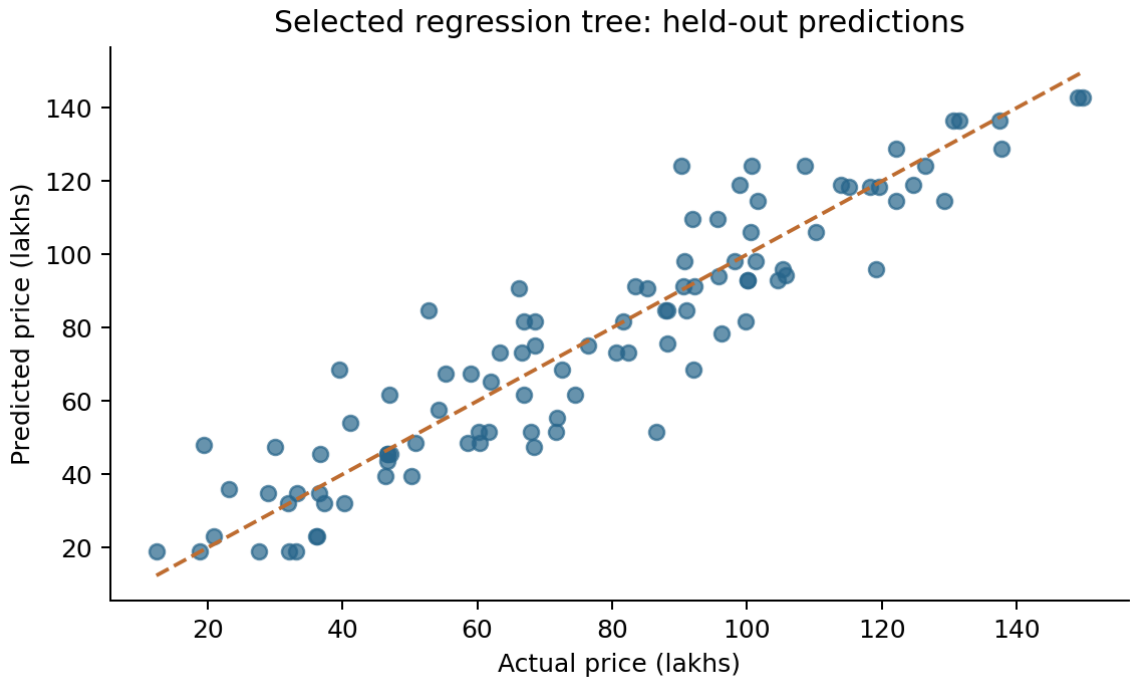


Figure 2. Actual versus predicted prices for the selected regression model.

The full tree has the lowest test RMSE (11.44 lakhs), although its CV RMSE (11.46) is worse than the pre-pruned tree's (10.82). It also achieves zero training RMSE with 300 leaves. The notebook reports the differing validation and test rankings transparently and retains the CV-selected model.

A hypothetical house of 1,800 square feet, three bedrooms, age five years, and distance 8 km receives a predicted price of 67.58 lakhs. The notebook checks that this value equals the mean training target in the reached leaf. This is a synthetic-data demonstration, not a market valuation.

5. Conclusions, limitations, and reproduction

5.1 Conclusions

CART provides interpretable models for both categorical and continuous targets. The classification model correctly labels 95.6% of the test wines using four leaves. The regression model captures substantial variation in the synthetic prices, with test R^2 of 0.858. Both outperform simple baselines.

Pruning controls complexity but does not guarantee improved performance on a particular test split. Cross-validation and test rankings differ in both experiments. The final model choices follow training validation rather than post-hoc selection of the best test score.

5.2 Limitations and further work

The wine dataset and its 45-sample test partition are small; one changed prediction moves accuracy by 2.22 percentage points. The housing simulation omits many real-world effects. Its results should not be generalized to actual property transactions.

Test results have been inspected during notebook development, especially the regression workflow. They are useful educational evaluations, not completely untouched external benchmarks. Independent data or nested cross-validation would support stronger claims.

Impurity importance can favor variables with more possible thresholds, and correlated inputs can complicate permutation importance. Neither establishes causality. Tree probabilities are leaf frequencies and are not automatically calibrated. Regression predictions are piecewise constant and do not extrapolate smoothly.

5.3 Reproduction and CSV loading

Open either notebook and run all cells in order. The notebooks recreate their datasets without external downloads and use seed 42. The CSV files are verified snapshots; the existing notebooks continue to generate/load their data internally.

For the classification CSV: read it with pandas.read_csv('Classification_Dataset.csv'); use all columns except Cultivar as X, and map class_0 → 0, class_1 → 1, class_2 → 2 for y. For regression, read Regression_Dataset.csv, use Price_lakhs as y, and the other four columns as X. Preserve row order and the documented split settings to reproduce the partitions.

Verified environment: Python 3.13.9; NumPy 2.2.6; pandas 2.3.3; scikit-learn 1.7.2; matplotlib 3.10.6. Package versions can affect numeric details.

5.4 Project sources and verification

Primary sources: Classification_Tree.ipynb and Regression_Tree.ipynb in the project root. Wine data provenance is documented in sklearn.datasets.load_wine().DESCR. The regression data-generation formula is fully specified in the regression notebook and in this report.

Both notebook workflows were rerun to prepare this report. All reported metrics and charts were generated from their executed variables. Both CSVs were reloaded and compared with the source DataFrames. No external or invented experimental results were used.